



*Advanced AI-based dynamic Integration
of heterogeneous data for semantic
clusters formalisation in human-centric
intelligent enterprise systems*

Benchmarking of SpaCy and BERT-like models

Mario Lezoche

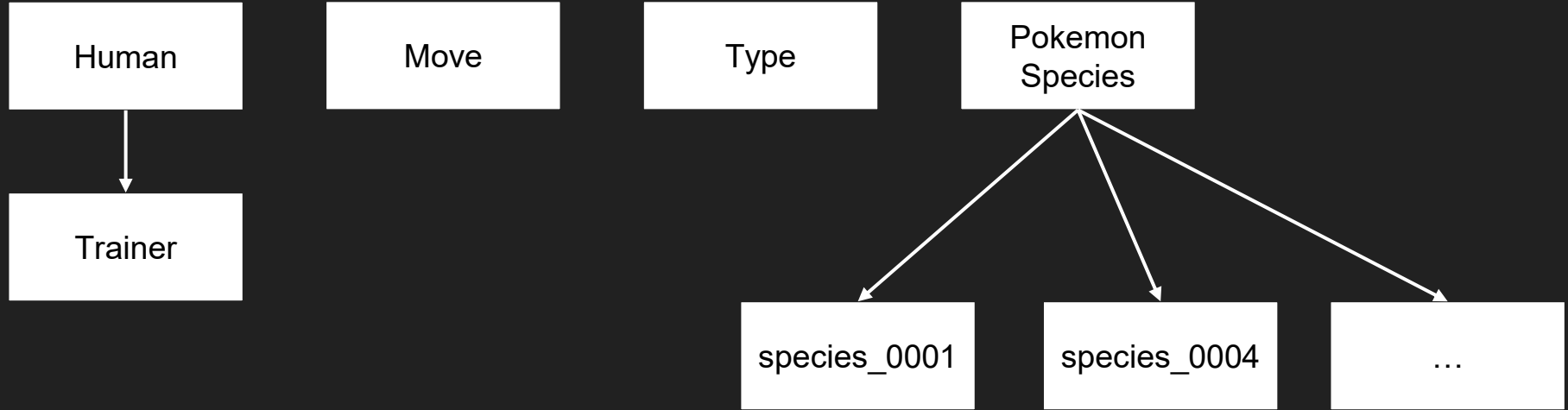
Thomas Pallet

Monday 02th February 2026

January summary

- Read chapter 5 & 6 of “*The Web of Data*”
- Build a mini ontology about Pokemon
- Done some NLP processing: txt -> classes (via NLP) -> translation into triples-> insertion in my OWL ontology
- Benchmark between SpaCy and BERT-like to do the same NLP pipeline

Pokemon ontology: Classes



A full-body illustration of Ash Ketchum, the main character of the Pokémon anime. He is a young boy with spiky black hair, wearing a red baseball cap, a blue vest over a white shirt, light blue jeans, and black and white sneakers. He is in a dynamic pose, with his right arm raised high holding a Poké Ball and his left fist clenched. He has a wide, enthusiastic smile.

		
Water Gun	Dragon Rage	Hyper Beam
		
Ice Beam	Skull Bash	Body Slam
		
Slam	Thunder	Thunderbolt

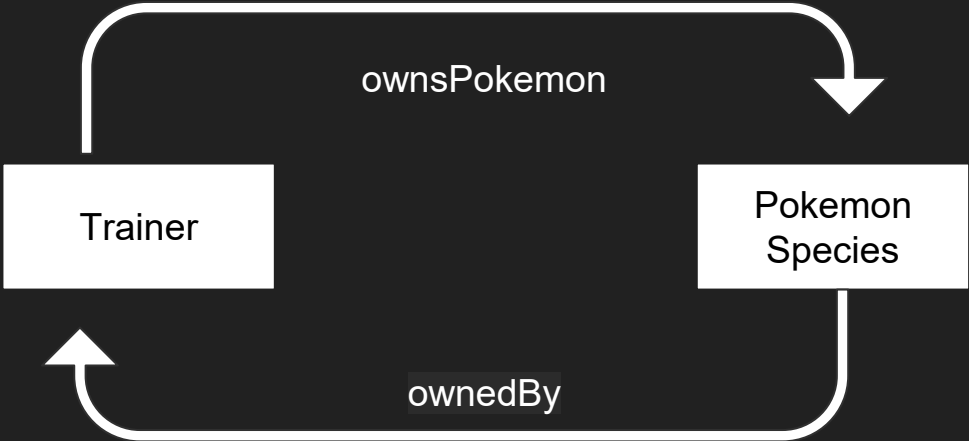
[illegible]

species_0025



Pokemon ontology: Example

Trainer



Pokemon
Species

species_0025



Table of content

01

**Creation of the
Dataset**

02

SpaCy model

03

**BERT-like
model**

04

Metrics

05

Results

06

Conclusion

Table of content

01

**Creation of the
Dataset**

02

SpaCy model

03

**BERT-like
model**

04

Metrics

05

Results

06

Conclusion

Generated text

To evaluate those approaches, generation of 10 texts/stories:

- Increasing difficulty :
 - explicit
 - implicit
 - long-distance
 - nested relations
- Multiple relations :
 - per sentence
 - correction and edit inside the text
 - indirect attributions
 - dense and ambiguous text.

Generated text

For each txt file :

- Built a JSON file with actual triples you can extract
- Relations follow the pokemon ontology

Generated text

Ash sets out with Pikachu beside him.
He stops to watch Bulbasaur among the vines.
Bulbasaur is a Grass and Poison type, at least in his notes.
In the same notebook, Pikachu is called an Electric type.
Ash writes that Pikachu can learn Thunder Shock.
He also writes that Thunder Shock is an Electric move.
Later, Bulbasaur uses Vine Whip to clear a path.
Vine Whip is listed as a Grass move in the journal.
Ash says Bulbasaur can learn Vine Whip.
He remembers that Bulbasaur evolves to Ivysaur.
A margin note adds that Ivysaur evolves from Bulbasaur.

Table of content

01

**Creation of the
Dataset**

02

SpaCy model

03

**BERT-like
model**

04

Metrics

05

Results

06

Conclusion

Entity typing

1. Extract labels from the OWL (given a dictionary of known names from the ontology)
1. Use EntityRuler object from spacy :
It tags words with the classes name like POKEMON_SPECIES, MOVE, TYPE, TRAINER
1. Now that entity has been set up, it can extract the relation because you know the type (domains + range)

Bulbasaur is a Grass and Poison type, at least in his notes.

Bulbasaur **POKEMON_SPECIES** is a **Grass TYPE** and **Poison TYPE** type, at least in his notes.

ACTUAL TRIPLES:

```
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Grass')  
( 'http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Poison')
```

PREDICTED TRIPLES:

```
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Grass')  
( 'http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Poison')
```

OK

Preprocessing - Unsupervised

- Tokenization: splits input text into words, punctuation (spaCy default)
- Sentence segmentation (line based)
- Lemmatization (base form for words)
- No stopwords, no stemming, no POS (part of speech = grouping per grammatical category) because it is an unsupervised model and rule based.

POS useful in supervised models because patterns are learned

Relation extraction (unsupervised)

- **Sentence-by-sentence extraction** with **rule patterns + type constraints**
 - a. **canLearnMove**: Pokémon + Move + keyword “**learn**”
→ clause-based pairing, **nearest entity** (≤ 10 tokens)
 - b. **hasType**: Pokémon + Type + keyword “**type**” or copula “**is**”
→ nearest entity (≤ 10 tokens)
 - c. **hasMoveType**: Move + Type + keyword “**move**” or “**which is**” or “**is**”
→ nearest entity (≤ 10 tokens)
 - d. **evolvesTo / evolvesFrom**: triggered by “**evolves to/from**”
→ subject = Pokémon **before** the phrase, object = Pokémon **after**
 - e. **ownsPokemon / ownedBy**: triggered by “**own**” / “**owned by**”
→ direction follows wording (Trainer $\leftrightarrow$ Pokémon)
- **Distance-based pairing**: relations are assigned by **token proximity** to reduce cross-product errors

Relation extraction (code)

```
# convenience for entity lookups
for sent in doc.sents:
    sent_ents = [e for e in doc.ents if e.start >= sent.start and e.end <= sent.end]
    by_type = defaultdict(list)
    for e in sent_ents:
        by_type[e.label_].append(e)

text = sent.text
# Pattern: X can Learn Y (distance-based + clause split)
if 'learn' in sent.text.lower():
    for clause in split_clauses(sent.text):
        clause_doc = nlp(clause)
        clause_ents = [e for e in clause_doc.ents if e.label_ in ('POKEMON_SPECIES', 'MOVE')]
        clause_pokes = [e for e in clause_ents if e.label_ == 'POKEMON_SPECIES']
        clause_moves = [e for e in clause_ents if e.label_ == 'MOVE']
        for m in clause_moves:
            nearest = None
            best = 999
            for p in clause_pokes:
                d = abs(p.start - m.start)
                if d < best and d <= 10:
                    best = d
                    nearest = p
            if nearest is not None:
                rels.append((nearest, 'canLearnMove', m, sent))

# Pattern: X is TYPE (distance-based + copula)
if 'type' in sent.text.lower() or 'is' in sent.text.lower():
    types = by_type.get('TYPE', [])
    pokes = by_type.get('POKEMON_SPECIES', [])
    for t in types:
        nearest = None
        best = 999
        for p in pokes:
            d = token_distance(p, t)
            if d < best and d <= 10:
                best = d
                nearest = p
        if nearest is not None:
            rels.append((nearest, 'hasType', t, sent))
```

```
# Pattern: MOVE is TYPE (distance-based + copula)
if 'move' in sent.text.lower() or 'which is' in sent.text.lower() or 'is' in sent.text.lower():
    moves = by_type.get('MOVE', [])
    types = by_type.get('TYPE', [])
    for t in types:
        nearest = None
        best = 999
        for m in moves:
            d = token_distance(m, t)
            if d < best and d <= 10:
                best = d
                nearest = m
        if nearest is not None:
            rels.append((nearest, 'hasMoveType', t, sent))

# Pattern: evolves to / from
if 'evolves to' in sent.text.lower():
    for p in by_type.get('POKEMON_SPECIES', []):
        for p2 in by_type.get('POKEMON_SPECIES', []):
            if p != p2 and p.start < p2.start:
                rels.append((p, 'evolvesTo', p2, sent))
if 'evolves from' in sent.text.lower():
    for p in by_type.get('POKEMON_SPECIES', []):
        for p2 in by_type.get('POKEMON_SPECIES', []):
            if p != p2 and p.start < p2.start:
                rels.append((p, 'evolvesFrom', p2, sent))

# Pattern: ownership
if 'own' in sent.text.lower() or 'owned by' in sent.text.lower():
    for tr in by_type.get('TRAINER', []):
        for p in by_type.get('POKEMON_SPECIES', []):
            if 'owned by' in sent.text.lower():
                # reverse direction
                rels.append((p, 'ownedBy', tr, sent))
            else:
                rels.append((tr, 'ownsPokemon', p, sent))
```

Outputs

Extracted relations

(subject_uri, predicate_uri, object_uri)

And outputs for the whole text to make evaluation of it:

```
story_01
Relations: 10
Extracted triples: 9
('http://example.org/pokemon#ThunderShock', 'http://example.org/pokemon#hasMoveType', 'http://example.org/pokemon#Electric')
('http://example.org/pokemon#VineWhip', 'http://example.org/pokemon#hasMoveType', 'http://example.org/pokemon#Grass')
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#canLearnMove', 'http://example.org/pokemon#VineWhip')
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#evolvesTo', 'http://example.org/pokemon#species_0002')
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Grass')
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Poison')
('http://example.org/pokemon#species_0002', 'http://example.org/pokemon#evolvesFrom', 'http://example.org/pokemon#species_0001')
('http://example.org/pokemon#species_0025', 'http://example.org/pokemon#canLearnMove', 'http://example.org/pokemon#ThunderShock')
('http://example.org/pokemon#species_0025', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Electric')
Actual triples: 9
TP: 9 (100.0% precision)
FP: 0
FN: 0 (100.0% recall)
F1: 100.0%
```

Outputs (examples)

Ash writes that Pikachu can learn Thunder Shock.

Ash **TRAINER** writes that Pikachu **POKEMON_SPECIES** can learn Thunder Shock **MOVE** .

ACTUAL TRIPLES:

```
('http://example.org/pokemon#species_0025', 'http://example.org/pokemon#canLearnMove', 'http://example.org/pokemon#ThunderShock')
```

PREDICTED TRIPLES:

```
('http://example.org/pokemon#species_0025', 'http://example.org/pokemon#canLearnMove', 'http://example.org/pokemon#ThunderShock')
```

OK

Bulbasaur is a Grass and Poison type, at least in his notes.

Bulbasaur **POKEMON_SPECIES** is a Grass **TYPE** and Poison **TYPE** type, at least in his notes.

ACTUAL TRIPLES:

```
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Grass')
```

```
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Poison')
```

PREDICTED TRIPLES:

```
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Grass')
```

```
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Poison')
```

OK

Outputs (examples)

Ash **TRAINER** opens a long entry that circles back on itself.

ACTUAL TRIPLES:

(none)

PREDICTED TRIPLES:

(none)

OK

He writes that Bulbasaur can learn Vine Whip, which is Grass, while Charmander can learn Ember, which is Fire.

He writes that Bulbasaur **POKEMON_SPECIES** can learn Vine Whip **MOVE**, which is Grass **TYPE**, while Charmander **POKEMON_SPECIES** can learn Ember **MOVE**, which is Fire **TYPE**.

ACTUAL TRIPLES:

```
('http://example.org/pokemon#Ember', 'http://example.org/pokemon#hasMoveType', 'http://example.org/pokemon#Fire')
('http://example.org/pokemon#VineWhip', 'http://example.org/pokemon#hasMoveType', 'http://example.org/pokemon#Grass')
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#canLearnMove', 'http://example.org/pokemon#VineWhip')
('http://example.org/pokemon#species_0004', 'http://example.org/pokemon#canLearnMove', 'http://example.org/pokemon#Ember')
('http://example.org/pokemon#species_0004', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Fire')
('http://example.org/pokemon#species_0004', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Grass')
```

PREDICTED TRIPLES:

```
('http://example.org/pokemon#Ember', 'http://example.org/pokemon#hasMoveType', 'http://example.org/pokemon#Fire')
('http://example.org/pokemon#VineWhip', 'http://example.org/pokemon#hasMoveType', 'http://example.org/pokemon#Grass')
('http://example.org/pokemon#species_0001', 'http://example.org/pokemon#canLearnMove', 'http://example.org/pokemon#VineWhip')
('http://example.org/pokemon#species_0004', 'http://example.org/pokemon#canLearnMove', 'http://example.org/pokemon#Ember')
('http://example.org/pokemon#species_0004', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Fire')
('http://example.org/pokemon#species_0004', 'http://example.org/pokemon#hasType', 'http://example.org/pokemon#Grass')
```

OK

Table of content

01

**Creation of the
Dataset**

02

SpaCy model

03

**BERT-like
model**

04

Metrics

05

Results

06

Conclusion

BERT Model (unsupervised)

1. Ontology based entity matching (Pokemon, Moves, ...)
Ontology labels as a dictionary, and the rule-based matching

1. Rule-based relation candidates
Detect words (like “learn”) to then mark the context as potential candidate for the relations.

```
relation_templates = {  
    'canLearnMove': 'can learn a move',  
    'hasType': 'has a type',  
    'hasMoveType': 'move type',  
    'evolvesTo': 'evolves to',  
    'evolvesFrom': 'evolves from',  
    'ownsPokemon': 'owns a pokemon',  
    'ownedBy': 'is owned by'  
}  
  
REL_EMB = {k: encode_text(v) for k,v in relation_templates.items()}
```

1. BERT sentence embedding as a semantic filter

```
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')  
model = AutoModel.from_pretrained('bert-base-uncased')
```

Rule-based relation candidates

```
def extract_relations(doc):
    rels = []

    for sent in doc.sents:
        sent_ents = [e for e in doc.ents if e.start >= sent.start and e.end <= sent.end]
        by_type = defaultdict(list)
        for e in sent_ents:
            by_type[e.label_].append(e)

        sent_text = sent.text
        sent_emb = encode_text(sent_text)
        def rel_ok(rel_name):
            return cosine(sent_emb, REL_EMB.get(rel_name, sent_emb)) >= REL_THRESH

        # canLearnMove
        if 'learn' in sent_text.lower() and rel_ok('canLearnMove'):
            for clause in split_clauses(sent_text):
                clause_doc = nlp(clause)
                clause_ents = [e for e in clause_doc.ents if e.label_ in ('POKEMON_SPECIES', 'MOVE')]
                clause_pokes = [e for e in clause_ents if e.label_ == 'POKEMON_SPECIES']
                clause_moves = [e for e in clause_ents if e.label_ == 'MOVE']
                for m in clause_moves:
                    nearest = None
                    best = 999
                    for p in clause_pokes:
                        d = abs(p.start - m.start)
                        if d < best and d <= 10:
                            best = d
                            nearest = p
                    if nearest is not None:
                        rels.append((nearest, 'canLearnMove', m, sent))
```

```
            rels.append((nearest, 'hasType', t, sent))

    # hasMoveType
    if (('move' in sent_text.lower() or 'which is' in sent_text.lower() or 'is' in sent_text.lower()) and rel_ok('hasMoveType')):
        moves = by_type.get('MOVE', [])
        types = by_type.get('TYPE', [])
        for t in types:
            nearest = None
            best = 999
            for m in moves:
                d = token_distance(m, t)
                if d < best and d <= 10:
                    best = d
                    nearest = m
            if nearest is not None:
                rels.append((nearest, 'hasMoveType', t, sent))

    # evolves to/from
    if 'evolves to' in sent_text.lower() and rel_ok('evolvesTo'):
        pokes = by_type.get('POKEMON_SPECIES', [])
        for p in pokes:
            for p2 in pokes:
                if p != p2 and p.start < p2.start:
                    rels.append((p, 'evolvesTo', p2, sent))
    if 'evolves from' in sent_text.lower() and rel_ok('evolvesFrom'):
        pokes = by_type.get('POKEMON_SPECIES', [])
        for p in pokes:
            for p2 in pokes:
                if p != p2 and p.start < p2.start:
                    rels.append((p, 'evolvesFrom', p2, sent))

    # ownership
    if 'owned by' in sent_text.lower() and rel_ok('ownedBy'):
        trainers = by_type.get('TRAINER', [])
        pokes = by_type.get('POKEMON_SPECIES', [])
        for tr in trainers:
            for p in pokes:
                rels.append((p, 'ownedBy', tr, sent))
    elif 'own' in sent_text.lower() and rel_ok('ownsPokemon'):
        trainers = by_type.get('TRAINER', [])
        pokes = by_type.get('POKEMON_SPECIES', [])
        for tr in trainers:
            for p in pokes:
                rels.append((tr, 'ownsPokemon', p, sent))
```

BERT Sentence embedding as a semantic filter

1. Encode the sentence with BERT, it gives us a vector
1. Encode a relation template (“can learn a move”), gives us a vector too
1. Compute cosine similarity (standard way to compare embeddings, measure direction similarity, ignoring the length)

$$\text{similarity}(A,B) = \frac{A \cdot B}{\|A\| \times \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n A_i^2} \times \sqrt{\sum_{i=1}^n B_i^2}}$$

1. And then keep the relation if similarity $\geq$ a threshold (heuristic approach) (0.15 in my case, manual trade-off)
- ⇒ So SpaCy is like ruled-based and BERT is like ruled-based + semantic gate (with vectorisation and embedding similarity)

Table of content

01

**Creation of the
Dataset**

02

SpaCy model

03

**BERT-like
model**

04

Metrics

05

Results

06

Conclusion

Evaluation metrics

In this case, relation is a classification problem at the triple level.

We compare then the actual vs predicted triples from each sentence from each story

- **TP (True Positive)**: predicted triple exists in actual
- **FP (False Positive)**: predicted triple not in actual
- **FN (False Negative)**: actual triple not predicted
- **Precision** = $TP / (TP + FP)$

“How many predicted relations are correct?”

- **Recall** = $TP / (TP + FN)$

“How many actual relations did we recover?”

- **F1** = $2PR / (P + R) \rightarrow$ important

Balance between precision and recall

Table of content

01

**Creation of the
Dataset**

02

SpaCy model

03

**BERT-like
model**

04

Metrics

05

Results

06

Conclusion

Results for SpaCy (unsupervised)

Story	TP	FP	FN	Precision (%)	Recall (%)	F1 (%)
story_01	9	0	0	100 %	100 %	100 %
story_02	10	0	0	100 %	100 %	100 %
story_03	11	0	0	100 %	100 %	100 %
story_04	13	0	1	100 %	92.9 %	96.3 %
story_05	17	0	0	100 %	100 %	100 %
story_06	17	0	0	100 %	100 %	100 %
story_07	15	1	1	93.8 %	93.8 %	93.8 %
story_08	15	0	0	100 %	100 %	100 %
story_09	17	0	0	100 %	100 %	100 %
story_10	24	0	0	100 %	100 %	100 %
TOTAL (mean)	-	-	-	99.38 %	98.67 %	99.01 % ²⁷

Results for BERT (unsupervised)

Story	TP	FP	FN	Precision (%)	Recall (%)	F1 (%)
story_01	8	1	0	88.9 %	100 %	94.1 %
story_02	9	1	0	90.0%	100 %	94.7 %
story_03	9	2	1	81.8 %	90.0 %	85.7 %
story_04	11	2	1	84.6 %	91.7 %	88.0 %
story_05	16	1	0	94.1 %	100 %	97.0 %
story_06	13	4	0	76.5 %	100 %	86.7 %
story_07	14	2	2	87.5 %	87.5 %	87.5 %
story_08	11	4	2	73.5 %	84.6 %	78.6 %
story_09	13	4	0	76.5 %	100 %	86.7 %
story_10	20	4	1	83.3 %	95.2 %	86.7 %
TOTAL (mean)	-	-	-	83.67 %	94.94 %	85.80 % 28

Results

spaCy gives a strong unsupervised baseline

- high precision on explicit relations
- performance drops as text complexity increases
- heavily adapted to the generated texts and rules

BERT-like unsupervised does not truly learn relations

- it mainly filters rule-based candidates
- performance depends on the same heuristic rules
- weak recall on implicit / long-distance relations

Table of content

01

**Creation of the
Dataset**

02

SpaCy model

03

**BERT-like
model**

04

Metrics

05

Results

06

Conclusion

Limitations

- No training $\Rightarrow$ cannot generalize beyond explicit patterns
- Rule-based extraction $\Rightarrow$ brittle on complex, nested, ambiguous sentences
- Over-linking or under-linking when multiple entities occur together

Improvements (expected) with supervised

- **BERT supervised** should improve recall on implicit and long-distance relations
- Better discrimination between true/false pairs
- More stable performance across story difficulty
- Less dependence on handcrafted rules

⇒ **But, need to do a dataset, annotation, preprocessing (quality of the data) and fine-tune it.**

Thank you for your attention !

Questions ?

Sources

SpaCy:

- [Implementing a custom trainable component for relation extraction](#)
- [Language Processing Pipelines · spaCy Usage Documentation](#)
- [Linguistic Features · spaCy Usage Documentation](#)

BERT-model:

- [Quickstart Hugging Face](#)
- [Sentence Embeddings using Siamese BERT-Networks - ACL Anthology](#)
- [\[1810.04805\] BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#)